StockSquares — Final Tech Lead Master Guide

System Architecture, Engineering Decisions, Team Leadership Framework, and Azure Deployment

Role Context: This dossier is prepared for the final Tech Lead executive interview. It covers four critical assessment vectors: **(1) System Architecture & Tech Stack:** Full-stack implementation using .NET 10, SQL Server 2022, React 19, Vite 6, and RxJS; **(2) Financial Correctness & Trade-offs:** Decimal math, serialized execution, transactional idempotency, and market simulator design; **(3) Tech Lead Team Management:** Delivery metrics, 70/20/10 capacity allocation, code reviews, and junior mentorship; **(4) Azure Cloud Deployment:** Complete hosting walkthrough across Azure Static Web Apps, Container Apps, and Azure SQL Serverless.

1. System Architecture & Clean Architecture Breakdown

The platform enforces strict Clean Architecture principles with inward dependency flow, ensuring financial rules and invariant execution remain completely decoupled from transport protocols, databases, or UI layers.

Layer	Project	Core Responsibilities & Technical Invariants
Domain	`StockSquares.Domain`	Pure domain entities (Order, Position, Portfolio, Trade). Strict financial precision using decimal. Encapsulates Weighted Average Cost Basis, Realized/Unrealized P&L; formulas, and matching evaluation. **Zero external dependencies** on EF Core, SQL, or HTTP.
Application	`StockSquares.Application`	Orchestrates use cases (Orders, Portfolio, Markets). Houses the TradingEngine serialized coordinator, DTO mapping, interface ports (IPriceFeedService, IRealtimePublisher), and canonical SHA-256 idempotency hashing.
Infrastructure	`StockSquares.Infrastructure`	Database persistence with EF Core 10 & SQL Server 2022 (TradingDbContext). Real-time SignalR WebSocket hub publisher. Hybrid ingestion: Binance Public WebSocket client with automatic fallback to an internal geometric Brownian motion generator.
API Host	`StockSquares.Api`	ASP.NET Core 10 Web API host, routing, Swagger/OpenAPI documentation, global correlation ID and exception handling middleware (mapping idempotency conflicts to HTTP 409), and CORS policies.
Frontend Web	`stocksquares-web`	React 19 SPA with TypeScript 5.7 (strict typing, zero any), Vite 6, and RxJS 7.8 reactive observable pipelines. BiDi Arabic RTL/LTR isolation, sub-second tick flash animations, interactive candlestick/area SVG charts, and cancellable HTTP requests.

2. Financial Invariants & Engineering Trade-Offs ("The Why")

During the interview, articulate the rationale behind each technical decision with commercial and financial precision:

- **High-Precision Monetary Math (Decimal vs Float):** Binary floating-point types (float, double) use IEEE 754 base-2 representation which cannot accurately represent common decimal fractions (e.g. 0.1 + 0.2 = 0.30000000000000004). In a trading engine, floating-point rounding leads to financial drift and reconciliation errors. All monetary fields (CashBalance, ReservedCash, AverageEntryPrice, RealizedPnL) strictly use C# 128-bit decimal configured to SQL decimal(18, 4) for prices/cash and decimal(18, 8) for crypto quantities.
- **Serialized Execution & Concurrency Safety:** To prevent race conditions, balance double-spending, and phantom executions, order validation, balance reservation, and tick-driven execution occur inside a serialized execution

coordinator (SemaphoreSlim) backed by atomic SQL Server transactions. Limit Buy orders lock the maximum required cash (Quantity * LimitPrice). When executed at a lower tick price (price improvement), excess reserved cash is instantly credited back to the available balance. Limit Sells reserve the exact asset quantity at order entry.

- **Transactional Idempotency & Conflict Detection (HTTP 409):** Network timeouts cause clients or frontends to resend requests. We persist an Idempotency-Key under a clustered primary key constraint in SQL Server. We deterministically compute a SHA-256 hash of the order payload (Symbol, Side, Type, Quantity, LimitPrice). An identical retry returns the cached OrderDto (safe deduplication). Reusing the same key with different parameters is caught via hash comparison and rejected with HTTP 409 Conflict (IDEMPOTENCY_CONFLICT).
- **Market-Tick Simulator vs. Full Double-Auction Order Book:** We deliberately implemented execution against real-world market ticks (Binance live feed) rather than an internal bid/ask double-auction order book. Retail paper traders practice by executing against true market liquidity. An internal order book for an evaluation would have no counterparty traders, leaving orders unfilled and creating an un-testable demonstration.
- **Hybrid Price Ingestion & Resilient Fallback:** Public crypto WebSocket streams can disconnect, get rate-limited, or be restricted by regional cloud firewalls. The BinanceWebSocketFeedService features a 10-second connection timeout that automatically falls back to an internal geometric Brownian motion generator, reconnecting in the background every 30 seconds. This guarantees 100% platform uptime and zero test failures.

3. Scaling to 1,000x: Architectural Blueprint

When asked how to scale from hundreds of users to 100,000+ concurrent traders and 50,000 orders/sec:

Partitioning by Symbol (Single-Writer Principle): Order book contention only happens for the same trading pair. By partitioning the engine across ticker symbols (BTCUSDT, ETHUSDT) using an LMAX Disruptor lock-free ring buffer, each symbol runs on an independent core with zero lock contention, achieving sub-microsecond matching.

CQRS & Event Sourcing: Transition from mutating relational rows to an Append-Only Event Log (OrderPlaced, OrderFilled, CashReserved). Commands execute in memory in microseconds; read models project asynchronously into Redis and SQL read replicas for sub-5ms GET /api/portfolio responses.

Azure SignalR Service Backplane: Offload in-process WebSocket connections to managed Azure SignalR Service in Serverless/Backplane mode. Client WebSockets terminate at edge gateways while backend nodes publish via Redis Pub/Sub, easily supporting 1,000,000+ simultaneous connections.

4. Tech Lead Leadership & Team Management Playbook

How to answer: *"How will you manage the team as a Tech Lead?"* Use the 3-Pillar framework to demonstrate executive maturity, structured agility, and high-performance engineering culture.

Pillar 1: Delivery & Agile Execution	Pillar 2: Technical Governance & Quality	Pillar 3: People, Culture & Mentorship
• **Vertical Slicing:** Slice user stories into thin, testable, end-to-end increments (UI to DB in 2-3 days) rather than horizontal layer silos. • **The 70/20/10 Rule:** 70% capacity for roadmap features, 20% for tech debt & refactoring, 10% for innovation & spikes. • **Definition of Done (DoD):** Code reviewed, 80%+ unit test coverage, integration tests verified, OpenAPI docs updated.	• **Empathy-Driven Reviews:** Separate blocking issues (correctness, financial risks, security) from non-blocking suggestions (style, nitpicks). • **Automate the Trivial:** Enforce linting (Oxlint) and formatting in pre-commit hooks and CI gates so human reviewers focus on architecture. • **Architectural Decision Records (ADRs):** Document major pivots (Context, Options, Decision, Consequences) transparently.	• **Empowering Engineers:** Delegate ownership of sub-modules to mid-level engineers with architectural guardrails. • **Psychological Safety:** Foster blameless post-mortems focusing on system safeguards rather than individual fault. • **Cross-Functional Bridge:** Translate commercial roadmaps into technical milestones; communicate technical constraints in business terms.

5. Azure Cloud Hosting Architecture & Deployment Guide

The application is optimized for Azure's Free and Serverless Tiers, satisfying enterprise cloud readiness without recurring costs.

Component	Azure Cloud Service	Configuration Details & Free Tier Entitlement
Frontend Web	Azure Static Web Apps	**Plan: Free ($0/mo)**. Global edge CDN, custom domains, free automatic SSL certificates, and integrated GitHub Actions push-to-deploy workflow.
Backend API	Azure Container Apps	**Plan: Consumption Serverless**. Free grant of 180,000 vCPU-seconds and 360,000 GiB-seconds per month. Automatic HTTPS ingress on port 5000, native WebSocket support.
Database	Azure SQL Database	**Plan: General Purpose - Serverless (Apply Free Limit)**. 32 GB storage + 100,000 vCore compute seconds free every month. Auto-pauses when inactive to save credit.
Container Registry	Azure Container Registry (ACR)	**Plan: Basic SKU**. Secure private Docker registry hosting stocksquares-api images.

Step-by-Step Azure Deployment Walkthrough

Step 1: Provision Azure SQL Database

1. In the Azure Portal (portal.azure.com), search for 'SQL Database' and click Create.
2. Create Resource Group rg-stocksquares, Database Name StockSquaresDb.
3. Create new Server (e.g. sql-stocksquares-prod), Authentication: SQL Authentication, User: cloudadmin, Password: YourStrongPass!2026.
4. Under Compute + Storage, select **General Purpose - Serverless** and check **'Apply Free Limit'**.
5. In Networking: Set **'Allow Azure services and resources to access this server'** to **YES**.
6. Copy your ADO.NET connection string from the database's Connection Strings tab.

Step 2: Deploy Backend (.NET 10 API on Container Apps)

1. In Azure Portal, search for 'Container Apps' and click Create.
2. Select Resource Group rg-stocksquares, Container App Name: app-stocksquares-api.
3. Environment: Create env-stocksquares in consumption mode.
4. Ingress: Enable Ingress = **Enabled**, Ingress traffic = **Accepting traffic from anywhere (External)**, Target Port = **5000**.
5. Environment Variables to add:
• ASPNETCORE_ENVIRONMENT = Production
• ASPNETCORE_HTTP_PORTS = 5000
• ConnectionStrings__DefaultConnection = *[Your Azure SQL Connection String]*
• CorsSettings__AllowedOrigins__0 = https://<your-frontend>.azurestaticapps.net (or *)
6. The API automatically runs ApplyMigrationsAndSeedAsync() on startup, creating tables and the $50,000 initial portfolio balance!

Step 3: Deploy Frontend (React 19 on Static Web Apps)

1. In Azure Portal, search for 'Static Web Apps' and click Create.
2. Plan type: **Free**. Deployment details: Sign in with GitHub and select repository TechLead, branch main.
3. Build Details: App location: stocksquares-web | Api location: *(empty)* | Output location: dist.
4. Click Review + Create. Azure generates a GitHub Actions workflow and compiles the app into a global URL.
5. In the Static Web App -> Configuration -> add Application Setting:
• VITE_API_URL = https://app-stocksquares-api.<region>.azurecontainerapps.io.
6. Open your frontend URL: SignalR status in the navbar will connect with a green badge!

CRITICAL AZURE GOTCHAS TO REMEMBER:
• **CORS Mismatch:** If your frontend URL is not configured in CorsSettings__AllowedOrigins__0 on the backend, browsers will block REST requests and SignalR WebSocket handshakes.
• **SQL Firewall:** If 'Allow Azure services to access server' is set to NO, the Container App will throw SQL connection timeouts.
• **WebSockets in Container Apps:** Supported natively. If deploying to Azure App Service instead, remember to toggle 'Web sockets: ON' under General Settings.

6. Tough Interview Questions & Battle-Tested Answers

Memorize and adapt these high-impact responses during your interview:

Q: Why didn't you use MediatR or CQRS immediately?

A: For an evaluation and system of this initial scope, direct typed domain services (OrderService, PortfolioService) provide direct call graphs, compile-time type safety, and zero indirection. MediatR introduces runtime reflection, boilerplate command/handler ceremonies, and harder-to-trace stack traces without adding architectural value at this scale. When scaling to event-sourced architectures with independent read/write data stores, CQRS becomes the right tool, which I outlined in Section 6 of the architecture blueprint.

Q: What happens if two users place an order at the exact same millisecond?

A: Order execution is serialized through our execution coordinator inside an atomic database transaction. Cash balance and asset reservations occur before order execution. If User A and User B submit simultaneous requests, the coordinator evaluates and reserves the required balance sequentially. Neither user can double-spend the balance. In a multi-replica production setup, we transition this from an in-memory SemaphoreSlim to SQL Server ROWVERSION concurrency tokens with Polly retry policies and Redis Redlock distributed locks partitioned by trader ID.

Q: How do you handle a developer on your team who resists writing unit tests?

A: I approach this with curiosity rather than authority. First, in a 1-on-1, I listen to understand their resistance—is the testing setup painful? Are mock setups too tedious? Second, I connect technical standards to developer experience and team velocity: Clean Architecture and unit tests aren't bureaucracy; they give us confidence to deploy on Fridays without breaking financial balances. Third, we establish shared agreements in team retrospectives, agree on pragmatic standards, and enforce them objectively through automated CI quality gates, taking personal ego out of the equation.

Q: How do you balance urgent business feature requests against paying down technical debt?

A: I use the 70/20/10 capacity allocation framework. We commit 70% of team capacity to delivering roadmap features, 20% dedicated to tech debt, automated testing, and performance optimization, and 10% for technical spikes. I explain technical debt to non-technical stakeholders in commercial terms: unaddressed tech debt acts like high-interest credit card debt; if we only pay the minimum balance, our future feature delivery slows to a crawl and incident frequency increases.

Q: How does your frontend handle high-frequency WebSocket price streams without lag?

A: Instead of triggering React re-renders on every tick, we use RxJS Observable pipelines. Incoming ticks update an in-memory market dictionary, and sub-second tick flash animations are handled via CSS transition decay classes rather than full DOM re-renders. Furthermore, toast notifications feature a 1,000ms deduplication buffer so traders aren't spammed during high-volatility market movements.